Physics Attestation Layer: A Physics-Based Digital Twin for Robot Arm Monitoring in Isaac Sim

Nov 2, 2026 · @Odysseus

This project builds a digital twin in NVIDIA Isaac Sim that audits a robot arm: a motion is trusted only when the torques it produces match the torques the commanded motion should produce.

1. Executive Summary

This project studies how a standard Franka Panda arm, simulated as a digital twin in NVIDIA Isaac Sim, loads its joints while it moves. The arm is run through different trajectories with different payloads, and its torque, velocity and related physics values are recorded and shown live at every joint angle.

The aim is to understand how torque changes with joint angle and payload, so that for a given payload the arm can follow a trajectory that takes as little time as possible.

2. Problem Statement

It is not clearly understood how torque is distributed across the joints of a Franka Panda as its joint angles and payload change during motion. Without this understanding, there is no firm basis for deciding how the arm should move along a given trajectory for a given payload.

The question this project addresses: for a specific payload and a specific trajectory, how should the arm move so that the motion time is as short as possible?

3. Motivation

The core idea is that **the control plane can be forged, but the physics cannot.** An attacker or a fault can change what a message says; it cannot change the laws that relate a joint's motion to the torque it needs. Checking motion against dynamics therefore adds a layer of defence that does not depend on trusting the message path.

Motivation	Why it matters
Security	Detects tampered or spoofed commands that pass authentication, at the physical layer
Safety	Flags unexpected loads, collisions, and abnormal effort before damage occurs
Maintenance	Reveals friction growth and looseness early, so repairs can be planned
Efficiency	Shows which motions waste torque and energy, so cycle time can be traded against wear
Explainability	Separates torque into gravity, acceleration, and coupling parts, so every alarm has a physical reason
Safe experimentation	A digital twin allows faults and attacks to be injected without risking hardware

Isaac Sim is chosen because its physics engine exposes the decomposed dynamics of an articulation (mass matrix, gravity term, Coriolis term, Jacobian, joint forces), which are rarely available from a real controller.

4. Objectives

The project delivers seven objectives. Each one produces a visible result.

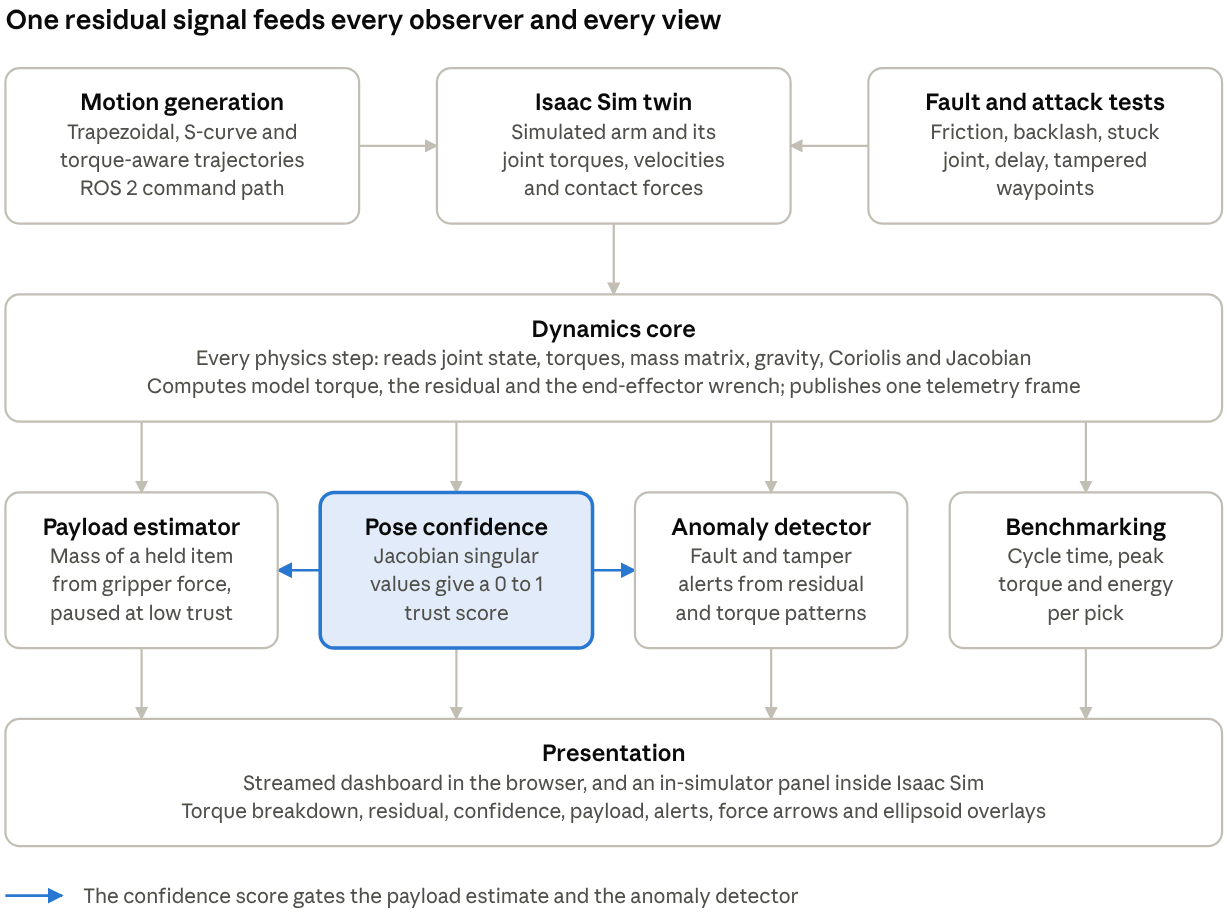
#	Objective	What it produces
O1	Read the arm's physics values every physics step	Live torque, velocity, force, mass-matrix, gravity, Coriolis and Jacobian data in one telemetry frame
O2	Compare required torque with measured torque	The residual, and its conversion to a force at the end effector
O3	Judge how trustworthy each pose is	A 0 to 1 confidence score that flags singular and near-singular poses
O4	Estimate the mass of an unknown held object	A payload estimate that converges live and is paused when confidence is low

#	Objective	What it produces
O5	Detect faults and tampered commands	Fault classes (friction, looseness, stuck joint, delay) and spoofed-setpoint alerts
O6	Compare movement styles	Cycle time, peak torque, and energy per pick for each trajectory type
O7	Present everything live	A streamed dashboard and an in-simulator panel with force and trajectory overlays

A supporting check validates the residual independently: the arm is switched to a low-gain torque mode, pushed with a known force, and the estimated end-effector force is compared with the applied one.

Dependencies. O1 and O2 are the foundation; if the model does not match the simulator on a slow, unloaded run, every later result would measure a bug rather than a physical effect. O3 gates O4 and O5. O6 and O7 consume the outputs of the others.

5. System Architecture



system architecture · 4 layers, 1 gate

The system has four layers, and nothing above the dynamics core touches the simulator directly.

- **Motion generation** produces the trajectories the arm executes. Fault and attack tests disturb these motions on purpose, so detection can be measured.
- **Dynamics core** is the foundation. It reads the simulator once per step and publishes a single telemetry frame that every other part consumes.
- **Observers** (payload estimator, pose confidence, anomaly detector, benchmarking) each read the same frame independently. Their only link is the confidence score, which gates the payload estimate and the detector.
- **Presentation** turns the frame and the observer results into the dashboard and the in-simulator view.

6. Theoretical Foundation

The project rests on the standard rigid-body dynamics of a serial robot arm. For joint angles q, the torque the joints must supply is:

$$\tau_{model} = M(q)\,\ddot{q} + C(q,\dot{q})\,\dot{q} + g(q)$$

- **M(q) q-double-dot** is the effort to accelerate the links. M is the mass matrix; it depends on pose because a stretched arm is harder to swing than a folded one.
- **C(q, q-dot) q-dot** is the effort caused by the links pulling on each other while moving (Coriolis and centrifugal coupling). It is small at low speed and grows quickly with speed.
- **g(q)** is the effort to hold the arm against gravity. On slow motions it is usually most of the torque.

The simulator also reports the torque each joint actually produced. The **residual** is the gap between the two:

$$r = \tau_{measured} - \tau_{model}$$

The Jacobian J(q) links joint space to the end effector. It converts the joint-space residual into an equivalent force and torque (a wrench) acting at the gripper:

$$F_{ext} = \left(J^{T}(q)\right)^{+} r$$

The quality of that conversion depends on how well-conditioned J is. Taking its singular values, the **confidence score** used throughout the project is the ratio of the smallest to the largest:

$$c(q) = \frac{\sigma_{min}(J)}{\sigma_{max}(J)}$$

A score near 1 means the pose is healthy; a score near 0 means the pose is close to singular and estimates must not be trusted.

How the residual is read.

Residual behaviour	Interpretation
Near zero	Arm moves as commanded, no extra load

Residual behaviour	Interpretation
Steady and offset	An unmodelled mass is attached, such as a gripped object
Sudden spike	Collision, fault, or a command inconsistent with the motion
Slow drift over many cycles	Wear, such as rising friction

The residual is minimised only once, during calibration, to prove the model matches the simulator. After that it is left untouched, because it is the signal being measured.

7. Implementation Approach

The system is built in layers so that each stage can be tested before the next depends on it. Each objective below lists the method, the inputs, and the output.

7.1 Telemetry capture (O1)

- Load the arm in Isaac Sim and wrap it as an articulation.
- In a callback that runs every physics step, read joint positions, velocities, measured efforts and applied efforts.
- Read the mass matrix, gravity term, Coriolis term and Jacobian from the articulation interface.
- Bundle everything with a timestamp into one flat telemetry frame, published every step.

7.2 Residual computation (O2)

- Estimate joint acceleration as the change in joint velocity between steps divided by the time step.
- Compute the model torque from the dynamics equation and subtract it from the measured torque.
- Convert the residual to an end-effector wrench with the pseudo-inverse of the transposed Jacobian.
- **Calibration test:** on a slow, unloaded trajectory the model must match the measurement within 5 percent per joint. Only then is the model accepted.

7.3 Pose confidence (O3)

- Take the singular value decomposition of the Jacobian at each step.
- Derive manipulability (product of singular values) and the confidence score (smallest divided by largest).
- Draw the velocity and force ellipsoids at the gripper; a flattened ellipsoid marks a near-singular pose.
- Below a set threshold the pose is marked low-trust and downstream estimates are paused or widened.

7.4 Payload estimation (O4)

- Grip an object of unknown mass and take the vertical component of the end-effector wrench.
- Mass is the vertical force divided by gravitational acceleration; the offset from the flange is found from the torque components.
- Smooth the estimate with a running filter and update it only when the confidence score is high.
- Validate against test masses of known value, and document the pose where the estimate becomes unobservable.

7.5 Fault and tamper detection (O5)

- **Inject mechanical faults** in the simulator: raised joint friction, backlash, a stuck joint, and delayed commands.
- **Inject command attacks:** altered waypoints and spoofed setpoints sent over the ROS 2 interface, including messages that pass authentication.
- Record residual and velocity data for normal runs and for each fault, and label them.
- Start with a threshold rule (residual above a limit for several consecutive steps), then train a small classifier such as a random forest on windowed features.
- Suppress alarms when the confidence score is low, so singular poses do not raise false alerts.
- Perturb link masses and add an unmodelled friction term so the twin is not identical to the simulated plant; otherwise detection would succeed trivially.

7.6 Trajectory benchmarking (O6)

- Generate three trajectories between identical waypoints: trapezoidal, S-curve (jerk-limited), and a smoother torque-aware profile.
- Run each and log cycle time, peak torque, and energy per pick.
- Energy is the sum over joints and steps of torque times velocity times time step; negative-power regions mark back-driven motion.
- Plot the three metrics against each other to find the best balance.

7.7 Independent validation

- Switch the arm to a low-gain torque mode and apply a known external force at the gripper.
- Compare the estimated end-effector force from the residual with the applied force.
- Agreement within tolerance confirms the residual mathematics without relying on any trained model.

8. Visualization and Demonstration

Results are shown in two places, fed by the same telemetry stream.

Streamed dashboard (browser). The telemetry frame is published over ROS 2 or a websocket to a Plotly Dash page. This mirrors a real plant monitoring stack and is the quickest to iterate on and record.

In-simulator view (Isaac Sim). An `omni.ui` panel docked in the viewport, with debug-draw overlays on the arm itself.

View	What it shows	Question it answers
Torque breakdown (stacked area per joint)	Gravity, acceleration and coupling parts of each joint's torque	Where does the effort go?
Residual trace with alarm band	Residual per joint and at the gripper	Is the arm behaving as commanded?
Confidence score trace	Pose quality over time, with low-trust zones shaded	Can the numbers be trusted right now?

View	What it shows	Question it answers
Payload estimate	Live mass estimate with its uncertainty band	What is the arm carrying?
Ellipsoid overlay at the gripper	Velocity and force ellipsoids drawn on the arm	Where is the arm strong, and where is it near-singular?
Link tint	Each link coloured by its share of the torque limit	Which joint is working hardest?
Force arrows	Estimated external force at the gripper	Is something pushing on the arm?
Trajectory comparison	Cycle time, peak torque, energy per pick for each movement style	Which motion is the best trade-off?
Alert panel	Fault class or tamper alert with the supporting residual	What went wrong, and why?

Demonstration scenario (one continuous run).

1. The arm picks up an unknown object and the payload estimate converges on the dashboard.
2. It carries the object through a near-singular pose; the confidence score dips and the estimate is visibly paused rather than wrong.
3. A tampered waypoint is sent over the ROS 2 interface. The message is accepted and the motion looks plausible, but the residual rises and an alert appears before the deviation is obvious to an observer.
4. A friction fault is then injected at one joint and identified by its torque signature.
5. The three trajectory styles are replayed side by side with their time, torque and energy figures.

9. Evaluation Plan

Each objective has a measurable pass condition.

Objective	Metric	Pass condition
O1 Telemetry	Frames published per physics step	No dropped frames across a full run
O2 Residual	Model torque against measured torque on a slow, unloaded run	Within 5 percent per joint
O3 Confidence	Estimate behaviour near singular poses	Estimates visibly suppressed, not reported as valid
O4 Payload	Error against known test masses	Convergence within a stated tolerance; unobservable case correctly flagged
O5 Detection	Detection rate and false-positive rate over a defined fault set	Reported with and without the confidence gate, to show the gate's effect
O6 Benchmark	Cycle time, peak torque, energy per pick	A comparison chart for all three trajectory styles
O7 Presentation	One uninterrupted demo run	Payload, singular pose and tamper alert shown in sequence
Validation	Applied force against estimated force in torque mode	Agreement within tolerance

For honest results, the twin's model is deliberately made slightly different from the simulated plant (altered link masses, extra friction). Detection and estimation are then reported against that mismatch, not against a perfect model.

10. Applications

The method is not tied to one robot or one simulator; any arm whose dynamics can be modelled and whose joint torques can be read can use it.

Area	Use
Industrial cybersecurity	A physical-layer intrusion check for robot cells, complementing network controls such as secured ROS 2 communication

Area	Use
Predictive maintenance	Early detection of friction growth, looseness and bearing wear from torque signatures
Pick-and-place and logistics	Weighing items in motion, and rejecting picks that carry the wrong load
Collaborative robots	Detecting unexpected contact from the residual, without extra force sensors
Process optimisation	Choosing motion profiles that cut peak torque and energy at an acceptable cycle time
Digital twin validation	Measuring how closely a twin's dynamics match the physical plant
Safe testing	Generating labelled fault and attack data without risking hardware
Education and research	A visual, explainable platform for teaching robot dynamics and anomaly detection

11. Risks and Limitations

Risk	Effect	Mitigation
Model matches the simulator too well	Detection looks better than it would on real hardware	Perturb link masses and add unmodelled friction deliberately
Acceleration estimated from velocity differences is noisy	Residual noise in fast motions	Filter velocity, and test first on slow motions
Singular poses	Wrong payload or alarm values	Confidence gate pauses estimates and alarms
Simulation to real gap	Friction and contact differ on a real arm	Report results as simulation findings; calibrate friction when moving to hardware

Risk	Effect	Mitigation
Scope is large	Several half-finished demos instead of one working system	Build in order: O1 and O2, then O3, O4 and O5, then O6 and O7
Attack coverage	Only the attack types that were injected are tested	State the tested fault and attack set explicitly in results

The system detects motion that is physically inconsistent with its command. An attack that produces a physically plausible motion toward an unsafe goal would not raise a residual; that case needs separate checks on the command's intent.

12. End Goal and Expected Outcomes

The end goal is a working, demonstrable digital twin that **audits** a robot arm from physics alone, and that knows when its own readings cannot be trusted.

On completion, the project will show:

- Live torque, velocity and force values for the arm, with torque split into gravity, acceleration and coupling parts.
- Evidence that the physics model matches the simulator on a calibration run.
- Poses where the arm is strong, where it struggles, and where readings become unreliable.
- The mass of an unknown held object, estimated from physics alone.
- Detection of mechanical faults and of tampered commands that look valid at the message level.
- A comparison of movement styles by speed, torque and energy.
- The arm's reaction to a known push, used to validate the method independently.
- One live dashboard and in-simulator view tying all of this together.

Delivered artefacts: the dynamics and telemetry module, the fault and attack injection scripts with their labelled dataset, the detector, the dashboard and in-simulator panel, and a results report with the evaluation table above filled in.

Build order. Phase 1: telemetry and residual, closed on a slow run. Phase 2: confidence score and payload estimation. Phase 3: fault and attack injection with detection. Phase 4: trajectory benchmarking, validation test and the full dashboard.